Wydział Elektroniki i Technik Informacyjnych
Politechnika Warszawska

Projektowanie układów sterowania (projekt grupowy)

Sprawozdanie z ćwiczenia laboratoryjnego nr 4, zespół nr 3

Michał Paradowski, Dominik Bijoch, Michał Dobrowolski

Warszawa, 2026

Spis treści

1. Laboratorium 2
1.1. Cel laboratorium 2
1.1.1. Stanowisko grzejąco-chłodzące 2
1.1.2. Stanowisko INTECO 2
1.2. Wyznaczenie punktu pracy 3
1.3. Mechanizm zabezpieczający stanowisko 3
1.4. Implementacja dwupętlowego regulatora PID 3
1.4.1. Dobór nastaw regulatorów PID 5
1.5. Pozyskiwanie odpowiedzi skokowych procesu 6
1.6. Implementacja regulatora DMC w wersji oszczędnej 7
1.6.1. Działanie regulatora DMC 10
1.7. Wizualizacja procesu na panelu operatorskim 10
1.8. Automat stanów modyfikujący wartości zadane 11

1. Laboratorium

1.1. Cel laboratorium

Celem ćwiczenia jest realizacja zadań z wykorzystaniem sterownika PLC oraz różnych oprogramowań (GX Works, GT Designer, MATLAB). Praca podzielona jest na dwa etapy:

1.1.1. Stanowisko grzejąco-chłodzące

1. **Weryfikacja komunikacji i punktu pracy:** Sprawdzenie możliwości sterowania sygnałami (G1, G2, W1, W2) i odczytu temperatur (T1, T3) oraz pomiar temperatur dla zadanych wartości sterowań.
2. **Implementacja bezpieczeństwa:** Zaprogramowanie mechanizmu wyłączającego grzałkę sąsiadującą z czujnikiem po przekroczeniu temperatury 250°C.
3. **Własny regulator PID:** Samodzielna implementacja (bez funkcji wbudowanych) dwupętlowego algorytmu PID na sterowniku, uwzględnienie ograniczeń sygnału i dobór nastaw.
4. **Regulator DMC:** Pozyskanie odpowiedzi skokowych, stworzenie modeli i implementacja analitycznego regulatora DMC 2×2 na sterowniku.
5. **Obsługa panelu operatorskiego:** Wyświetlenie na panelu wartości zadanych, mierzonych i sterujących.
6. **Automat stanów:** Implementacja algorytmu sekwencyjnego zmieniającego wartości zadane w co najmniej 3 krokach.

1.1.2. Stanowisko INTECO

1. **Zabezpieczenie stanowiska:** Implementacja mechanizmu chroniącego przed uszkodzeniem (np. przepełnieniem, wychyleniem).
2. **Identyfikacja:** Próba wyznaczenia charakterystyki statycznej procesu.
3. **Własny regulator PID:** Implementacja i dostrojenie autorskiego algorytmu PID.
4. **Wbudowany regulator PID:** Uruchomienie i konfiguracja gotowej funkcji PID sterownika oraz porównanie wyników z implementacją autorską.
5. **Automat stanów:** Oprogramowanie sekwencyjnej zmiany wartości zadanych (min. 3 stany).
6. **Zaawansowana wizualizacja:** Przygotowanie szczegółowej reprezentacji graficznej procesu, wykresów (sygnały wyjściowe, zadane, sterujące) oraz wizualizacji grafu przejść automatu.

1.2. Wyznaczenie punktu pracy

Podczas realizacji zadania wykonano następujące kroki:

1. Uruchomiono komunikację ze stanowiskiem (GXWorks oraz Matlab) i sprawdzono poprawność odczytu rejestrów D100 - T_1 oraz D101 - T_3, D114 - G_2, D115 - G_2, a następnie poprzez watch ustawiono wentylatory (D110 - W1 oraz D111 - W3) na 50%,
2. Ustalono wartość sygnałów sterujących grzałką zgodnie z punktem pracy $G_1 = 25 + \mathbf{3} \bmod 5 = 28$ oraz $G_2 = 30 + \mathbf{3} \bmod 5 = 33$.
3. Pozostawiono układ w stanie ustalonym i odczytano wartość temperatury **T1** oraz **T3**.

Uzyskane pomiary potwierdziły poprawne działanie stanowiska. Odczytano temperature $Y_1 = T_1 = 32{,}9\,°\mathrm{C}$ oraz $Y_2 = T_3 = 35{,}4\,°\mathrm{C}$

1.3. Mechanizm zabezpieczający stanowisko

W celu ochrony stanowiska przed uszkodzeniem fizycznym oraz skutkami awarii czujników temperatury, zaimplementowano na sterowniku PLC mechanizm bezpieczeństwa. Algorytm działa w trybie nadrzędnym w stosunku do algorytmów regulacji.

Zgodnie z treścią zadania, krytyczna wartość temperatury została ustalona na poziomie 250°C. Ponieważ odczyty temperatury w rejestrach sterownika przechowywane są jako liczby całkowite przeskalowane o współczynnik 100 (tj. 250,00°C odpowiada wartości 25000), warunek logiczny sprawdza przekroczenie tej wartości.

Logika została rozdzielona na dwa niezależne tory, aby w przypadku awarii jednego czujnika wyłączana była tylko grzałka z nim sąsiadująca, co pozwala na ewentualną diagnostykę drugiego toru. Ponadto w warunku zerowane są również wszystkie zmienne pomocnicze wykorzystywane w regulatorach.

```
IF D100 >= 25000 THEN
    D114 := 0;       (* Natychmiastowe wyłączenie grzałki G1 *)
    u1_k := 0;       (* Wyzerowanie zmiennej sterującej regulatora dla toru 1 *)
    u1_k_1 := 0.0;
    u1_k_new := 0.0;
    dUp1[0] := 0.0;
END_IF;

IF D110 >= 25000 THEN
    D115 := 0;       (* Natychmiastowe wyłączenie grzałki G2 *)
    u2_k := 0;       (* Wyzerowanie zmiennej sterującej regulatora dla toru 2 *)
    u2_k_1 := 0.0;
    u2_k_new := 0.0;
    dUp2[0] := 0.0;
END_IF;
```

Listing 1.1. Kod mechanizmu bezpieczeństwa zaimplementowany na sterowniku

Powyższy kod umieszczono na końcu głównej pętli z regulatorami (w Fixed Scan), co gwarantuje, że nawet jeśli regulator obliczy niezerową wartość sterowania, zostaną one nadpisane wartością bezpieczną (0) przed wysłaniem sygnału na wyjścia fizyczne.

1.4. Implementacja dwupętlowego regulatora PID

Własną implementację dwupętlowego regulatora PID rozpoczęto od zapewnienia deterministycznego czasu próbkowania. Cel ten zrealizowano poprzez umieszczenie kodu algorytmu w sekcji `Fixed Scan` oraz na ustawieniu okresu przerwania wywołującego program na 4000 ms.

Parametry algorytmu dyskretnego (współczynniki r_n) wyznaczono na podstawie nastaw ciągłych (K, T_i, T_d) w trybie offline za pomocą skryptu w środowisku MATLAB:

```
Tp = 4; % Czas probkowania (Sampling time)
% Nastawy dla PID 1
K1  = 0.5;
Ti1 = 35;
Td1 = 3;
% Nastawy dla PID 2
K2  = 0.5;
Ti2 = 35;
Td2 = 3;
% Definicja wzorow (funkcje anonimowe dla czytelnosci)
calc_r0 = @(K, Ti, Td) K * (1 + Tp/(2*Ti) + Td/Tp);
calc_r1 = @(K, Ti, Td) K * ((Tp/(2*Ti)) - (2*Td/Tp) - 1);
calc_r2 = @(K, Ti, Td) K * Td/Tp;
% Wyswietlenie wynikow dla PID 1
fprintf('// --- Nastawy PID 1 ---\n');
fprintf('r0_PID1 := %.5f;\n', calc_r0(K1, Ti1, Td1));
fprintf('r1_PID1 := %.5f;\n', calc_r1(K1, Ti1, Td1));
fprintf('r2_PID2 := %.5f;\n', calc_r2(K1, Ti1, Td1));
```

Obliczone współczynniki regulatora zostały zaimplementowane w sekcji `INIT` sterownika, co wynika z braku potrzeby ich cyklicznej aktualizacji w czasie pracy algorytmu:

```
// --- Nastawy PID 1 ---
r0_PID1 := 0.90357;
r1_PID1 := -1.22143;
r2_PID1 := 0.37500;
// --- Nastawy PID 2 ---
r0_PID2 := 0.90357;
r1_PID2 := -1.22143;
r2_PID2 := 0.37500;
```

Kolejnym krokiem jest odczyt aktualnych wartości sygnałów wyjściowych z rejestrów oraz obliczenie uchybów regulacji dla obu pętli:

```
y1_k := INT_TO_REAL(D100);
y2_k := INT_TO_REAL(D101);

// Przesuniecie historii uchybow
e1_k_2 := e1_k_1;
e1_k_1 := e1_k;
e1_k := y1_zad - y1_k;
e2_k_2 := e2_k_1;
e2_k_1 := e2_k;
e2_k := y2_zad - y2_k;
```

Następnie wyznaczono przyrosty sterowania $\Delta u(k)$ (zmienne `u_k_new`) zgodnie z dyskretnym równaniem regulatora PID:

```
u1_k_new := r0_PID1*e1_k + r1_PID1*e1_k_1 + r2_PID1*e1_k_2;
u2_k_new := r0_PID2*e2_k + r1_PID2*e2_k_1 + r2_PID2*e2_k_2;
```

Zastosowano ograniczenia wartości sterowania, a następnie wyznaczono ostateczne wartości sygnałów sterujących $u_j(k)$:

```
// Kanal 1 - ograniczenie i wyznaczenie sterowania
IF (u1_k_1 + u1_k_new) > 1000.0 THEN
    u1_k_new := 1000.0 - u1_k_1;
ELSIF (u1_k_1 + u1_k_new) < 0.0 THEN
    u1_k_new := 0.0 - u1_k_1;
END_IF;

// Kanal 2 - ograniczenie i wyznaczenie sterowania
IF (u2_k_1 + u2_k_new) > 1000.0 THEN
    u2_k_new := 1000.0 - u2_k_1;
ELSIF (u2_k_1 + u2_k_new) < 0.0 THEN
    u2_k_new := 0.0 - u2_k_1;
END_IF;
```

```
// Sumowanie koncowe
u1_k := u1_k_1 + u1_k_new;
u2_k := u2_k_1 + u2_k_new;
```

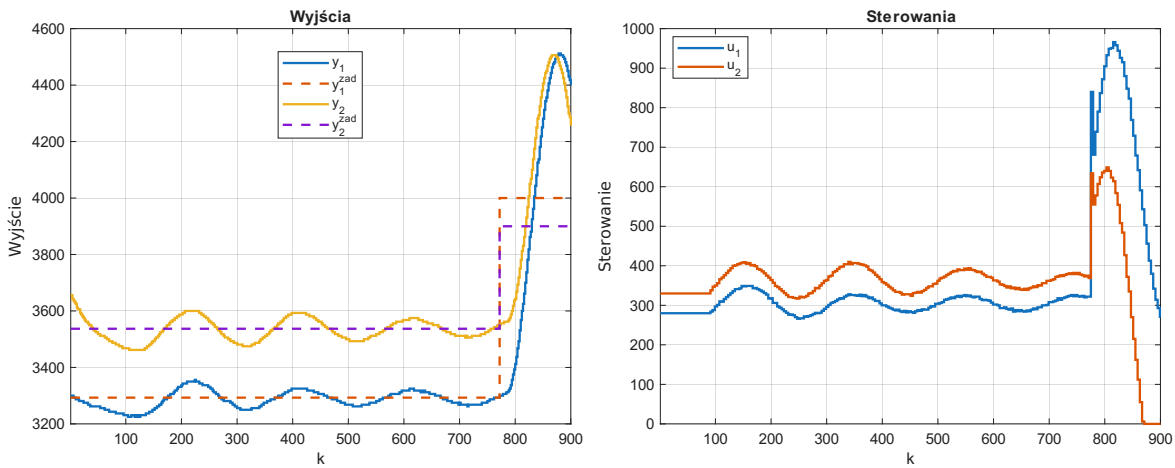
Ostatnim etapem jest przekazanie wyznaczonych wartości sterowań do obiektu poprzez zapisanie ich w odpowiednich rejestrach wyjściowych po uprzedniej konwersji typów danych:

```
D114 := REAL_TO_INT(u1_k);
D115 := REAL_TO_INT(u2_k);
```

1.4.1. Dobór nastaw regulatorów PID

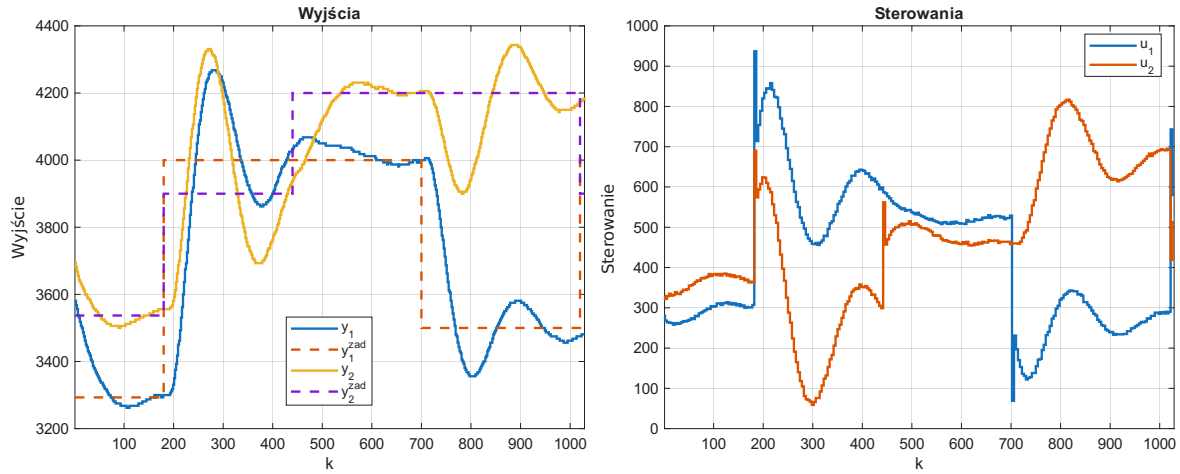
Nastawy regulatorów PID zostały dobrane metodą eksperymentalną. Ze względu na zbliżoną charakterystykę dynamiczną obu kanałów procesowych, zdecydowano się na zastosowanie identycznych parametrów dla obu pętli regulacji.

W pierwszym etapie strojenia przyjęto wstępne wartości parametrów: $K = 0{,}4$, $T_i = 20$ oraz $T_d = 3$. Wyniki regulacji dla tych nastaw przedstawiono na rysunku 1.1.



Rys. 1.1. Przebiegi sygnałów wyjściowych oraz sterujących dla wstępnych nastaw PID

Powyższe parametry skutkowały niską jakością regulacji, objawiającą się występowaniem słabo tłumionych oscylacji sygnału wyjściowego. W celu poprawy odpowiedzi układu dokonano korekty nastaw, zwiększając wzmocnienie oraz czas zdwojenia. Nowe parametry wyniosły odpowiednio: $K = 0{,}5$, $T_i = 35$ oraz $T_d = 3$.



Rys. 1.2. Przebiegi sygnałów wyjściowych oraz sterujących dla skorygowanych nastaw PID

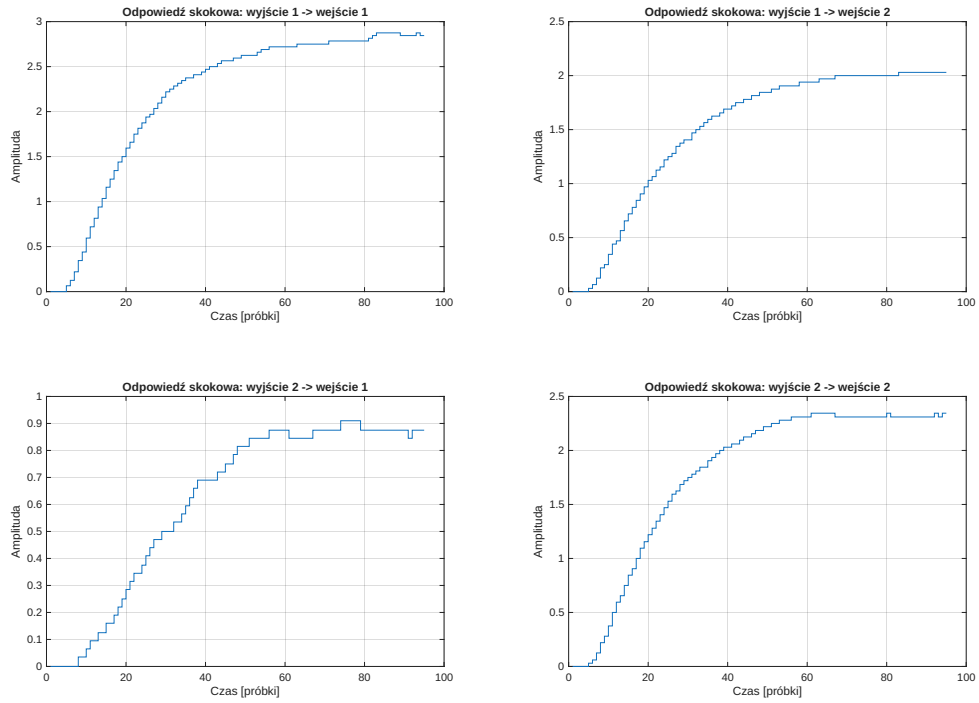
Analiza przebiegów zamieszczonych na rysunku 1.2 pozwala stwierdzić poprawę jakości regulacji. Układ charakteryzuje się większą stabilnością, a sygnał wyjściowy osiąga wartość zadaną w akceptowalnym czasie. Mimo dużych przeregulowań oraz występujących oscylacji, można zauważyć, że w skończonym czasie regulator PID jest w stanie ustabilizować wyjścia na wartośći zadanej. Niestety w przypadku skoku dwóch wartości zadanych na raz, radzi on sobie gorzej niż w przypadku pojedynczej zmiany.

1.5. Pozyskiwanie odpowiedzi skokowych procesu

Na potrzeby implementacji regulatora DMC, zebrano odpowiedzi skokowe procesu na obu torach sterowania. W tym celu zebrano dane ze stanowiska a następnie zastosowano kod Matlab.

```
load('l4/data/step_resp_u1_28_48.mat')
u1_new1 = u1(21:400);
u2_new1 = u2(21:400);
y1_new1 = y1(21:400);
y2_new1 = y2(21:400);

load('l4/data/step_resp_u2_33_53.mat')

u1_new2 = u1(6:385);
u2_new2 = u2(6:385);
y1_new2 = y1(6:385);
y2_new2 = y2(6:385);

du = 200;

SS = zeros(385-5, 2, 2);

SS(:, 1, 1) = (y1_new1-y1_new1(1))/du;
SS(:, 1, 2) = (y2_new1-y2_new1(1))/du;
SS(:, 2, 1) = (y1_new2-y1_new2(1))/du;
SS(:, 2, 2) = (y2_new2-y2_new2(1))/du;

SS_new = SS(1:4:end, :, :);
```

Uzyskano w ten sposób 4 jednostkowe odpowiedzi skokowe. Odpowiedzi te, zaprezentowano na rys 1.3.



Rys. 1.3. Odpowiedzi skokowe wszystkich torów, dla stanowiska grzewczo-chłodzącego

1.6. Implementacja regulatora DMC w wersji oszczędnej

Własną implementację regulatora DMC w wersji oszczędnej rozpoczęto od zapewnienia deterministycznego czasu próbkowania. Cel ten zrealizowano poprzez umieszczenie kodu algorytmu w sekcji `Fixed Scan` oraz na ustawieniu okresu przerwania wywołującego program na 4000 ms.

Parametry algorytmu dyskretnego (współczynniki K_e oraz K_u) wyznaczono na podstawie uzyskanych wcześniej odpowiedzi skokowych oraz parametrów regulatora w trybie offline za pomocą skryptu w środowisku MATLAB:

```
load('data/SS.mat');
SS = SS(1:4:end, :, :);
D = size(SS, 1);
N = 20;
Nu = 5;
nu = 2;
ny = 2;

psi_vec = ones(ny*N, 1);
Psi = diag(psi_vec);

lambda_vec = ones(Nu*nu, 1)*3;
Lambda = diag(lambda_vec);

M_tensor = zeros(N, Nu, ny, nu);
for i = 1:N
    for j = 1:Nu
        if i < j
            M_tensor(i, j, :, :) = zeros(ny, nu);
        else
            M_tensor(i, j, :, :) = SS(i - j + 1, :, :);
        end
    end
end
```

```
M_perm = permute(M_tensor, [3, 1, 4, 2]);
M = reshape(M_perm, ny*N, nu*Nu);

K = (M'*Psi*M + Lambda)\(M'*Psi);

K1 = K(1:nu, :);

Ke = zeros(nu, ny);
for p = 1:N
    idx_start = (p-1)*ny + 1;
    idx_end   = p*ny;
    K1_p = K1(:, idx_start:idx_end);
    Ke = Ke + K1_p;
end

Ku = cell(D-1, 1);

for i = 1:D-1
    Mp_i = zeros(ny*N, nu);

    S_i = squeeze(SS(i, :, :)); % ny x nu

    for p = 1:N
        idx_row_start = (p-1)*ny + 1;
        idx_row_end   = p*ny;

        if (p + i) > D
            S_future = squeeze(SS(D, :, :));
        else
            S_future = squeeze(SS(p + i, :, :));
        end

        Mp_i(idx_row_start:idx_row_end, :) = S_future - S_i;
    end

    Ku{i} = K1 * Mp_i; % nu x nu
end

fid = fopen('dmc_params.txt', 'w');
fprintf(fid, 'Ke11 := %.6f;\n', Ke(1,1));
fprintf(fid, 'Ke12 := %.6f;\n', Ke(1,2));
fprintf(fid, 'Ke21 := %.6f;\n', Ke(2,1));
fprintf(fid, 'Ke22 := %.6f;\n', Ke(2,2));
for i = 1:D-1
    fprintf(fid, 'Ku11[%d] := %.6f; ', i-1, Ku{i}(1,1));
end
fprintf(fid, '\n');
for i = 1:D-1
    fprintf(fid, 'Ku12[%d] := %.6f; ', i-1, Ku{i}(1,2));
end
fprintf(fid, '\n');
for i = 1:D-1
    fprintf(fid, 'Ku21[%d] := %.6f; ', i-1, Ku{i}(2,1));
end
fprintf(fid, '\n');
for i = 1:D-1
    fprintf(fid, 'Ku22[%d] := %.6f; ', i-1, Ku{i}(2,2));
end
fprintf(fid, '\n');
fclose(fid);
```

Obliczone współczynniki regulatora zostały skopiowane do sekcji `INIT` sterownika, co wynika z braku potrzeby ich cyklicznej aktualizacji w czasie pracy algorytmu:

```
    Ke11 := 0.382111;
    Ke12 := -0.140072;
    Ke21 := 0.064562;
    Ke22 := 0.400857;
    Ku11[0] := 0.227949; Ku11[1] := 0.239726; // I dak dalej aż do Ku11[378]
    Ku12[0] := 0.016164; Ku12[1] := 0.022746; // I tak dalej aż do Ku12[378]
    Ku21[0] := 0.016128; Ku21[1] := 0.022702; // I tak dalej aż do Ku21[378]
    Ku22[0] := 0.228154; Ku22[1] := 0.239464; // I tak dalej aż do Ku22[378]
```

Kolejnym krokiem jest odczyt aktualnych wartości sygnałów wyjściowych z rejestrów oraz obliczenie uchybów regulacji dla obu pętli:

```
y1_k := INT_TO_REAL(D100);
y2_k := INT_TO_REAL(D101);

// Przesuniecie historii uchybow
e1_k_2 := e1_k_1;
e1_k_1 := e1_k;
e1_k := y1_zad - y1_k;
e2_k_2 := e2_k_1;
e2_k_1 := e2_k;
e2_k := y2_zad - y2_k;
```

Następnie wyznaczono przyrosty sterowania $\Delta u(k)$ (zmienne `u_k_new`) zgodnie z równaniamy algorytmu DMC w wersji oszczędnej:

```
// Obliczenie nowej zmiany sterowania (Część proporcjonalna)
u1_k_new := Ke11*e1_k + Ke12*e2_k;
u2_k_new := Ke21*e1_k + Ke22*e2_k;

// Pętla DMC (Horyzont dynamiki)
FOR licznik := 0 TO 378 BY 1 DO
    u1_k_new := u1_k_new - Ku11[licznik]*dUp1[licznik] + Ku12[licznik]*dUp2[licznik];
    u2_k_new := u2_k_new - Ku21[licznik]*dUp1[licznik] + Ku22[licznik]*dUp2[licznik];
END_FOR;
```

Zastosowano ograniczenia wartości sterowania, a następnie wyznaczono ostateczne wartości sygnałów sterujących $u_j(k)$ i zaktualizowano przeszłe wartości zmiany sterowań na potrzeby dalszej regulacji:

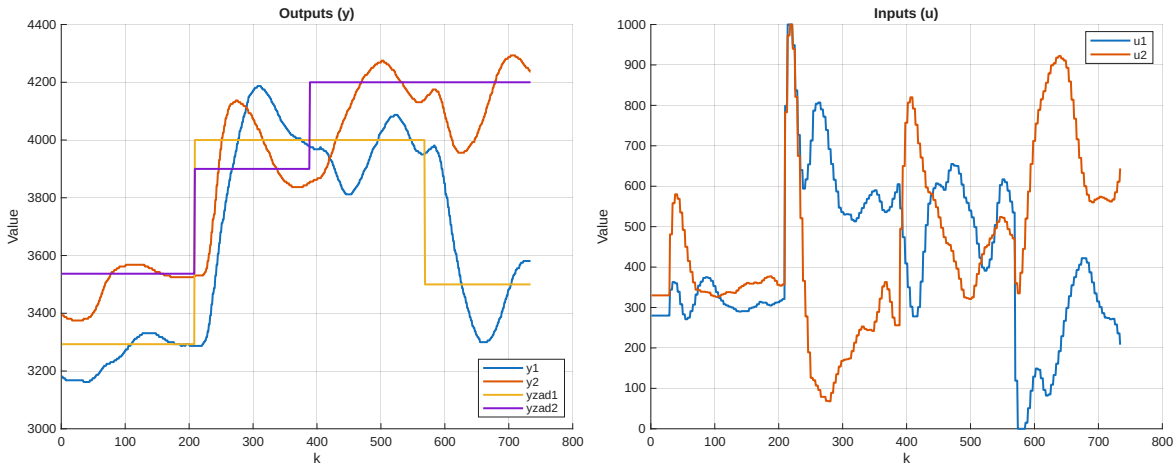
```
// Kanal 1 - ograniczenie i wyznaczenie sterowania
IF (u1_k_1 + u1_k_new) > 1000.0 THEN
    u1_k_new := 1000.0 - u1_k_1;
ELSIF (u1_k_1 + u1_k_new) < 0.0 THEN
    u1_k_new := 0.0 - u1_k_1;
END_IF;

// Kanal 2 - ograniczenie i wyznaczenie sterowania
IF (u2_k_1 + u2_k_new) > 1000.0 THEN
    u2_k_new := 1000.0 - u2_k_1;
ELSIF (u2_k_1 + u2_k_new) < 0.0 THEN
    u2_k_new := 0.0 - u2_k_1;
END_IF;

// Sumowanie koncowe
u1_k := u1_k_1 + u1_k_new;
u2_k := u2_k_1 + u2_k_new;

// Przesunięcie historii przyrostów sterowania
FOR licznik := 378 TO 1 BY -1 DO
    dUp1[licznik] := dUp1[licznik-1];
    dUp2[licznik] := dUp2[licznik-1];
END_FOR;

dUp1[0] := u1_k_new;
dUp2[0] := u2_k_new;
```

Ostatnim etapem jest przekazanie wyznaczonych wartości sterowań do obiektu poprzez zapisanie ich w odpowiednich rejestrach wyjściowych po uprzedniej konwersji typów danych:

```
D114 := REAL_TO_INT(u1_k);
D115 := REAL_TO_INT(u2_k);
```

1.6.1. Działanie regulatora DMC

Parametry regulatora DMC ustawiono zgodnie z intuicją oraz doświadczeniem nabytym w trakcie poprzednich laboratoriów - możliwie małe N oraz N_u. Parametr λ zdecywano ustawić się na 3, natomiast parametr ψ zostawiono na 1 dla obydwu wyjść, gdyż są to bardzo podobne grzałki, z podobnym wpływem na siebie. Finalnie dobrane parametry regulatora DMC $N = 20, N_u = 5, \Psi =$ `diag(ones(ny*N, 1))`, $\Lambda =$ `diag(ones(Nu*Nu, 1))` $\cdot\, 3$. Przykładowy przebieg regulacji przedstawiono na rysunku 1.4



Rys. 1.4. Przebiegi sygnałów wyjściowych oraz sterujących dla algorytmu DMC

Biorąc pod uwagę charakter stanowiska grzewczo-chłodzącego oraz wysoką podatność na zakłócenia, regulator radził sobie ze sprowadzaniem wartości wyjściowych do wartośći zadanych przez automat stanów opisany w jednej z następnych sekcji. Analiza odpowiedzi skokowej wykazuje, że układ regulacji dąży do osiągnięcia wartości zadanej dla obu wyjść. Należy odnotować, że pomimo rygorystycznych ograniczeń czasowych wynikających z konfiguracji automatu stanów, odpowiedź układu charakteryzuje się gasnącymi oscylacjami, co potwierdza zachowanie stabilności procesu.

Z uwagi na ograniczony czas laboratorium oraz chęć realizacji drugiej części laboratorium z dźwigiem Inteco, powyższy przebieg, z intuicyjnie dobranymi parametrami, uznano za akceptowalny i z tego powodu nie przeprowadzano dalszego dostrajania regulatora DMC.

1.7. Wizualizacja procesu na panelu operatorskim

Celem wizualizacji było umożliwienie operatorowi bieżącego podglądu stanu obiektu, w tym wartości mierzonych, zadanych oraz sygnałów sterujących dla obu pętli regulacji (pętla temperatury T_1 oraz T_3). Zrzut ekranu zaimplementowanego panelu przedstawiono na rys. 1.5.

Rys. 1.5. Widok ekranu panelu operatorskiego z wizualizacją procesu.

Główny ekran wizualizacji został podzielony na strefy odpowiadające poszczególnym zmiennym procesowym. Ze względu na symetryczność obiektu (dwie grzałki, dwa czujniki), zastosowano bliźniaczy układ wyświetlania danych dla obu torów sterowania.

Na ekranie wyświetlane są następujące parametry liczbowe:

— **YZAD1** oraz **YZAD2** – wartości zadane temperatur (odpowiednio dla T_1 i T_3 dla każdej grzałki). Na przedstawionym zrzucie wynoszą one odpowiednio 45.00 oraz 37.00.
— **Y1** oraz **Y2** – aktualne wartości mierzone temperatur (zmienne procesowe). Wartości widoczne na ekranie to 32.81°C oraz 35.16°C.
— **U1** oraz **U2** – wartości sygnałów sterujących mocą grzałek G_1 i G_2 (zmienne sterujące), wyrażone w procentach od 0 do 100.

Wartości te zostały dodatkowo zwizualizowane za pomocą wykresów słupkowych, znajdujących się w centralnej części ekranu. Każdej pętli przypisano zestaw trzech słupków, co pozwala na szybką ocenę uchybu regulacji:

— Słupek czerwony: obrazuje wartość zadaną.
— Słupek zielony: obrazuje wartość mierzoną.
— Słupek żółty: obrazuje poziom wysterowania grzałki.

1.8. Automat stanów modyfikujący wartości zadane

W celu automatyzacji procesu weryfikacji układu regulacji, zaimplementowano na sterowniku automat stanów.

Logika została zrealizowana w języku ST przy użyciu instrukcji `CASE`. Przejścia między kolejnymi stanami odbywają się cyklicznie w oparciu o timer systemowy `TIM_MAIN` typu TON, skonfigurowany na czas trwania kroku wynoszący 320 sekund. Automat modyfikuje wartości zadane temperatury dla obu torów ($y1_zad$ oraz $y2_zad$). Automat umieszczono w sekcji Scan w pliku MAIN.

Zaimplementowano następującą sekwencję zmian:

— **Stan 1:** Ustawienie punktu pracy na $T_1 = 40{,}00$°C oraz $T_3 = 39{,}00$°C.
— **Stan 3:** Skokowa zmiana wartości zadanej w torze drugim do $T_3 = 42{,}00$°C.
— **Stan 4:** Skokowa zmiana wartości zadanej w torze pierwszym do $T_1 = 35{,}00$°C.

Na listingu 1.2 przedstawiono kod realizujący opisaną logikę. Wartości w kodzie są przeskalowane o mnożnik 100 (np. 4000 odpowiada 40,00°C).

```
TIM_MAIN(PT:= T#320s);

CASE Stan_MAIN OF
    0:  (* Inicjalizacja *)
        Stan_MAIN := 1;
        TIM_MAIN.IN := 1;
    1:  (* Krok 1: Ustalenie punktu pracy 40C / 39C *)
        IF TIM_MAIN.Q THEN
            y1_zad := 4000.0;
            y2_zad := 3900.0;
            TIM_MAIN.IN := 0;  (* Reset timera *)
            Stan_MAIN := 2;    (* Przejście do nast. stanu *)
        END_IF;
    2:  (* Krok 2: Skok T3 na 42C *)
        TIM_MAIN.In := 1;
        IF TIM_MAIN.Q THEN
            y2_zad := 4200.0;
            TIM_MAIN.IN := 0;
            Stan_MAIN := 3;
        END_IF;
    3:  (* Krok 3: Skok T1 na 35C *)
        TIM_MAIN.In := 1;
        IF TIM_MAIN.Q THEN
            y1_zad := 3500.0;
            TIM_MAIN.IN := 0;
            Stan_MAIN := 0;
        END_IF;
END_CASE;
```

Listing 1.2. Implementacja automatu stanów w języku ST